Junit e Copertura

Elvinia Riccobene - Valentina Ciriani

Junit

Tool utilizzato in laboratorio

Per avere tutto già predisposto in una versione di eclipse stand-alone per windows (senza bisogno di altre istallazioni), scaricare:

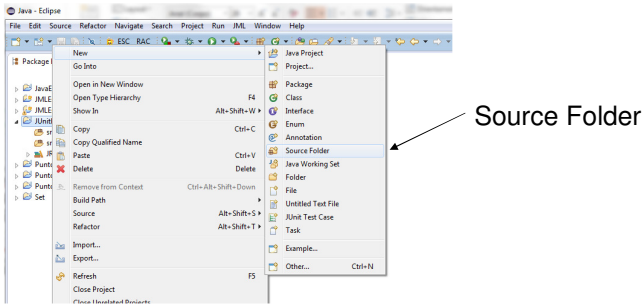
http://fmse.di.unimi.it/sw/eclipsePSS_win.zip

Scompattare il file zip ed eseguire eclipse_java7.bat (non eclipse.exe!)

L'installazione contiene anche JUnit ed Eclemma che useremo per la parte di testing

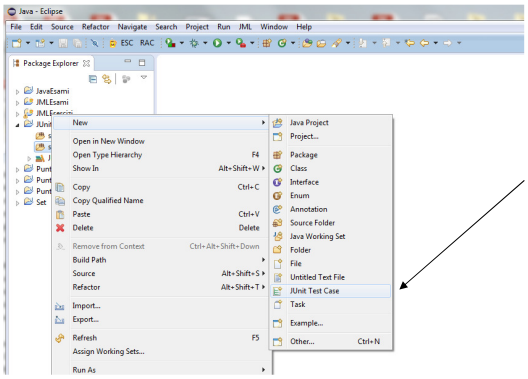
Come creare test case

- Creare un nuovo progetto java
- Creare un nuovo source folder (es. src_test) che conterrà i test case



Come creare test case

- Creare un JUnit Test Case in src_test
- Aggiungendo JUnit 4 nel build path



JUnit Test Case

Test case

- Si inizia con un test set vuoto del tipo:

```
import static org.junit.Assert.*;

public class Test {

   @Test
   public void test() {
    fail("Not yet implemented");
   }

}
```

Primo test

- Deve iniziare sempre con `@Test`

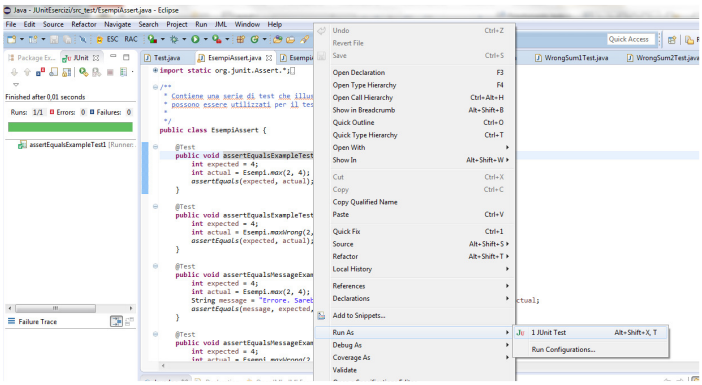
```
import static org.junit.Assert.*;

public class Test {

   @Test
   public void test1() {
    …
   }

}
```


Esempi

- Scaricare dal sito del corso i file
  - classiDatestare.zip
  - testDiEsempio.zip
- Estrarre il contenuto e copiare:
  - le classi da testare in src
  - I test di esempio in src_test
- EsempiAssert.java
- EsempiAssertArrayEquals.java

Come usare JUnit

Prendere il nome del caso di test da considerare
e far Run As Junit Test



Esempi

- Scaricare dal sito del corso i file
  - SumClass.java
  - testSum.zip
- Estrarre il contenuto e copiare:
  - le classi da testare in src
  - I test di esempio in src_test

Esercizi per casa

Costruire i test (JUnit) per gli esercizi:

- Knight's tour
- Sokoban

Nel file eserciziJUnit.pdf

Coperture ed Eclemma

Tipo di coperture

1. Copertura delle **istruzioni** (statment): ogni istruzione deve essere eseguita almeno una volta
2. Copertura delle **decisioni** (equivalente alla copertura degli archi o branch): per ogni decisione ci deve essere un caso di test che la renda vera e un caso di test che la renda falsa
3. Copertura delle **condizioni**: per ogni condizione deve esistere un test che la renda vera e un test che la renda falsa

Esempio `if (a<3 && b==4)` ....

Condizione

Decisione

Operatori logici

- Tra valori di tipo boolean è possibile utilizzare operatori logici:
- & AND
- | OR
- ! NOT
- ^ XOR
- && AND lazy
- || OR lazy

Operatore lazy: &&

- A && B
  - valuta prima A e, se necessario, valuta in seguito B
  - ovvero, valuta A:
    - se A ha valore false restituisce false
    - se A ha valore true restituisce il valore di B
- Es (2==0) && (x!=3) (2==2) && (x!=3)

In rosso le espressioni valutate

- A & B
  - E' l'AND standard tra A e B: sono valutati sempre tutti e due
- Es (2==0) & (x!=3) (2==2) & (x!=3)

Operatore lazy: ||

- A || B
  - valuta prima A e, se necessario, valuta in seguito B
  - ovvero, valuta A:
    - se A ha valore true restituisce true
    - se A ha valore false restituisce il valore di B
- Es (2==2) || (x!=3) (2==0) || (x!=3)

In rosso le espressioni valutate

- A | B
  - E' l'OR standard tra A e B: sono valutati sempre tutti e due
- Es (2==2) | (x!=3) (2==0) | (x!=3)
- perché non esiste lo XOR lazy?

Esempio

1. if (x!=0 && y/x > 8) y++;
2. if (x!=0 & y/x > 8) y++;

- Qual è la differenza tra questi due comandi?
- Il primo è sempre corretto perché
  - nel caso di x =0 la divisione y/x non viene effettuata
- Il secondo da un errore in esecuzione (eccezione)
  - quando x=0, perché calcola y/0!
- In pratica il primo è equivalente a due if annidati

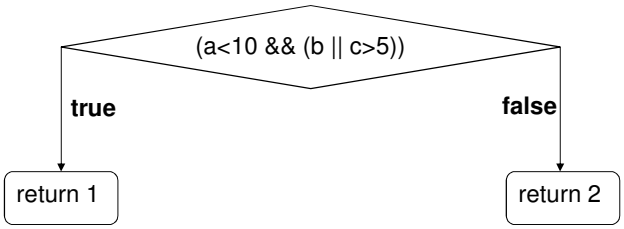
```
if (x!=0)
    if (y/x > 8) y++;
```


Esempi

- Aprire un nuovo progetto con src e src_test
- Costruire un nuovo file di tipo JUnit test case
  - La copertura è sempre in un file di tipo Junit test case
- Scaricare dal sito del corso i file
  - EsempioPerCoverageConLazy.java
  - coperturaEsempioPerCoverageConLazy.zip
- Estrarre il contenuto e copiare:
  - le classi da testare in src
  - I test per la copertura in src_test

Coperture con operatori lazy

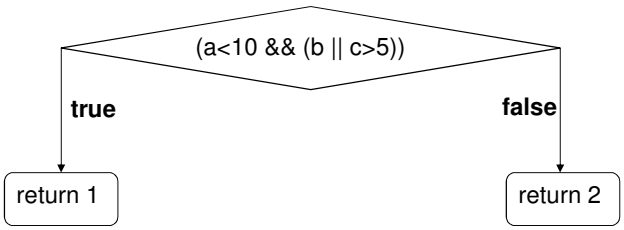
```
public static int foo(int a, boolean b, int c) {
   if (a < 10 && (b || c > 5)) {
        return 1;
   }
   else {
        return 2;
   }
}
```



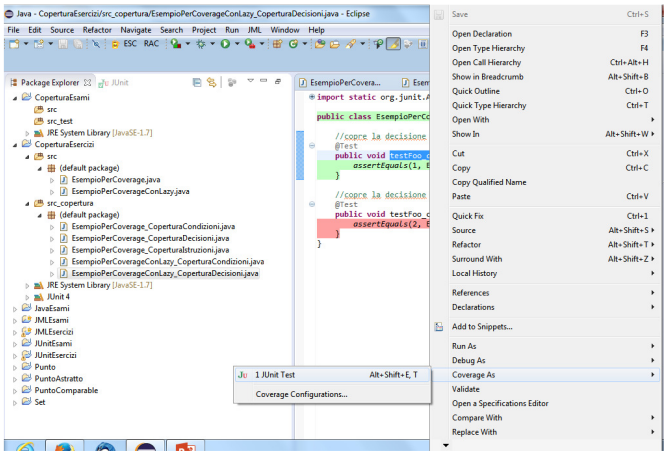
Copertura con operatori lazy

Copertura **Decisioni** (in questo esempio è uguale alla copertura delle **Istruzioni**):

1) (a<10 && (b || c>5)) true
2) (a<10 && (b || c>5)) false



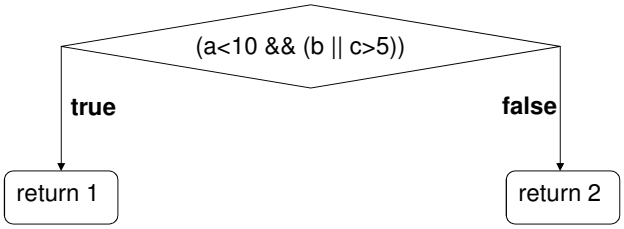
Uso di Eclemma



Copertura con operatori lazy

Copertura **Condizioni**: (a<10 && (b || c>5))

1) "a < 10" a true, "b" a true (lazy evaluation non copre "c > 5")
2) "a < 10" a true "b" a false e "c > 5" a true
3) "a < 10" a true "b" a false e "c > 5" a false
4) "a < 10" a false (lazy evaluation non copre "b" e "c > 5")

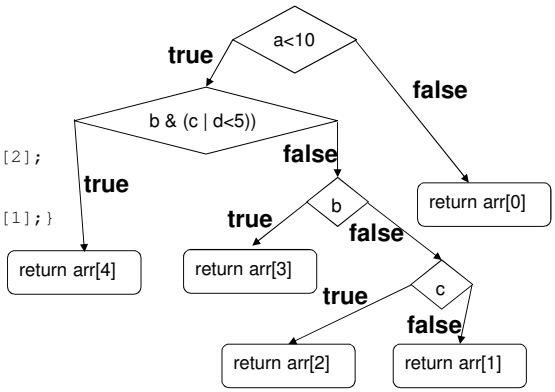


Esempi

- Scaricare dal sito del corso i file
  - EsempioPerCoverage.java
  - coperturaEsempioPerCoverage.zip
- Estrarre il contenuto e copiare:
  - le classi da testare in src
  - I test per la copertura in src_test

Coperture con operatori non lazy

```
public public static int foo(int a, boolean b, boolean c, int d, int[] arr) {
   if (a < 10) {
        if (b & (c | d < 5))
            return arr[4];
        else
            if (b)
                return arr[3];
            else
                if (c)
                    return arr[2];
                else
                    return arr[1];}
   else
        return arr[0];
}
```



Esercizio

Costruire i test (Copertura con Eclemma) per l'esercizio:

- TicTacToe

Nel file **esercizioCopertura.pdf**

Da finire a casa